

# LAB 07 — PASSWORD ANALYSIS: JOHN THE RIPPER

Credential Cracking, Hash Analysis & ATT&CK Mapping

CySA+ Home Lab Portfolio | Alejandro W. Orellana | June 28, 2026

## 1. Lab Overview

This lab demonstrates offensive credential analysis techniques using John the Ripper, one of the most widely used password auditing tools in penetration testing and red team operations. The objective is to simulate an attacker obtaining a Linux shadow file and cracking password hashes — then map those techniques to MITRE ATT&CK; to understand the defensive implications.

| Field              | Detail                                                          |
|--------------------|-----------------------------------------------------------------|
| Tool               | John the Ripper 1.9.0-jumbo-1+bleeding-aec1328d6c (OMP)         |
| Target             | Local Linux shadow file — 3 test accounts (MD5/crypt \$1\$)     |
| Environment        | Kali Linux — VirtualBox isolated lab                            |
| Hash Format        | md5crypt / crypt(3) \$1\$ — MD5-based Linux password hashing    |
| Attack Modes       | Wordlist (dictionary) attack + Incremental (brute force) attack |
| ATT&CK; Techniques | T1003 (OS Credential Dumping), T1110.002 (Password Cracking)    |

## 2. Environment Setup

Three test user accounts were created with intentionally weak passwords to simulate real-world credential hygiene failures. Passwords were hashed using MD5 (openssl passwd -1) to ensure John the Ripper compatibility with the jumbo build on Kali.

| Username  | Password    | Hash Format      | Risk Level                                   |
|-----------|-------------|------------------|----------------------------------------------|
| testuser1 | password123 | \$1\$ (MD5crypt) | CRITICAL — Top 10 most common password       |
| testuser2 | abc123      | \$1\$ (MD5crypt) | CRITICAL — Extremely common, rockyou top 100 |
| testuser3 | letmein     | \$1\$ (MD5crypt) | CRITICAL — Dictionary word, instant crack    |

## 3. Attack Phase 1 — Credential Extraction (T1003)

The unshadow utility combines /etc/passwd and /etc/shadow into a single John-compatible file. In a real attack scenario, this step requires root-level access — simulating privilege escalation followed by credential dumping.

Command executed:

```
$ sudo unshadow /etc/passwd /etc/shadow > /tmp/crackme.txt
$ grep "testuser" /tmp/crackme.txt
```

Output (shadow file entries extracted):

```
testuser1:$1$QAQRNfi$V5f7BN0b3Ek2Zp8q2AtLU0:1002:1002:~/home/testuser1:/bin/sh
testuser2:$1$ThS0Qo1i$cCgLYzeca0QCCNgSsQq2G0:1003:1003:~/home/testuser2:/bin/sh
testuser3:$1$IvRProTh$6WG7aAi0ZepVCqQ3Nu0Vq0:1004:1004:~/home/testuser3:/bin/sh
```

**Analyst Note:** The \$1\$ prefix confirms MD5crypt hashing. Each hash includes a unique salt (e.g. \$QAQRNfi\$) which prevents precomputed rainbow table attacks — but does not protect against dictionary or brute force attacks against weak passwords.

#### 4. Attack Phase 2 — Wordlist / Dictionary Attack (T1110.002)

John the Ripper's wordlist mode tests each password hash against a dictionary of known passwords. The rockyou.txt wordlist (14.3 million entries) was used — the industry standard for password auditing derived from the 2009 RockYou data breach.

Command executed:

```
$ john --wordlist=/usr/share/wordlists/rockyou.txt /tmp/crackme.txt
```

Output:

```
Using default input encoding: UTF-8
Loaded 3 password hashes with 3 different salts (md5crypt, crypt(3) $1$)
Will run 4 OpenMP threads
abc123 (testuser2)
letmein (testuser3)
password123 (testuser1)
3g 0:00:00:00 DONE (2026-06-28 10:58) 20.00g/s 10240p/s 17920c/s
Session completed.
```

| Metric            | Value                       | Significance                                         |
|-------------------|-----------------------------|------------------------------------------------------|
| Passwords cracked | 3 of 3 (100%)               | All test accounts compromised                        |
| Time to crack     | Under 1 second              | Instant — passwords found in rockyou top entries     |
| Crack speed       | 20,000 g/s                  | 20,000 password guesses per second on MD5crypt       |
| CPU threads       | 4 (OpenMP)                  | John auto-scaled to all available cores              |
| Wordlist used     | rockyou.txt (14.3M entries) | Industry standard — sourced from 2009 RockYou breach |

#### 5. Attack Phase 3 — Incremental (Brute Force) Mode

Incremental mode systematically generates and tests every possible character combination rather than relying on a pre-built wordlist. This demonstrates the fundamental tradeoff between attack coverage (brute force finds everything eventually) and speed (wordlist is orders of magnitude faster against common passwords).

### Output (session aborted after 30 seconds):

```
Loaded 3 password hashes with 3 different salts (md5crypt, crypt(3) $1$)
Will run 4 OpenMP threads
abc123 (testuser2)
1g 0:00:00:51 0.01945g/s 15452p/s 31159c/s
Session aborted
```

| Mode        | Speed      | Passwords Cracked | Time                 | Coverage                                   |
|-------------|------------|-------------------|----------------------|--------------------------------------------|
| Wordlist    | 20,000 g/s | 3 of 3 (100%)     | < 1 second           | Limited to wordlist contents               |
| Incremental | 15,452 p/s | 1 of 3 in 51 sec  | 51 seconds (aborted) | Exhaustive — finds any password given time |

## 6. MITRE ATT&CK; Mapping

| Technique ID | Name                           | Tactic                         | Lab Activity                                                                                                                              |
|--------------|--------------------------------|--------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| T1003        | OS Credential Dumping          | Credential Access              | unshadow combined /etc/passwd + /etc/shadow into crackable format — simulates attacker extracting credentials after root access           |
| T1110.002    | Brute Force: Password Cracking | Credential Access              | John the Ripper run in both wordlist mode (rockyou.txt) and incremental brute force mode against extracted MD5crypt hashes                |
| T1078        | Valid Accounts                 | Persistence / Lateral Movement | Successfully cracked credentials represent valid accounts an attacker could use for authenticated access — no further exploitation needed |

## 7. Analyst Findings & Defensive Implications

**Weak passwords fall instantly to dictionary attacks:** All three test passwords (password123, abc123, letmein) were cracked in under one second using the rockyou wordlist. These are among the most commonly used passwords globally — any attacker with basic tooling would crack them immediately after obtaining a shadow file.

**Salted hashing slows brute force but does not prevent dictionary attacks:** MD5crypt uses unique salts per password, which defeats rainbow table lookups. However, salting provides zero protection against dictionary attacks — if the password exists in rockyou.txt, the salt is irrelevant. This demonstrates why password complexity requirements matter independently of hashing strength.

**Brute force is dramatically slower but exhaustive:** Incremental mode cracked abc123 in 51 seconds vs under 1 second in wordlist mode — a 50x+ slowdown. Against truly random passwords (e.g. 12+ character mixed case with symbols), brute force would take years on a single machine. This is the computational argument for strong password policies.

**Credential dumping requires root access — privilege escalation is the prerequisite:** The unshadow step requires sudo/root to read /etc/shadow. In a real attack chain, an attacker would first need to exploit a privilege escalation vulnerability before reaching this step — connecting this lab directly to the vulnerability findings in Lab 03 (Metasploitable2 had multiple CVSS 9.8-10.0 findings that could enable root access).

**Key Takeaway:** Password cracking is not a sophisticated attack — it is a fast, automated process that requires only a shadow file and a wordlist. The defensive controls are well-established: enforce minimum 12-character passwords with complexity requirements, implement account lockout policies, use modern hashing (bcrypt/Argon2 over MD5), and monitor for /etc/shadow access attempts (auditd rule: -w /etc/shadow -p r -k shadow\_read).

## 8. Recommended Defensive Controls

| Control                               | Implementation Guidance                                                                                                                                    |
|---------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Password Policy</b>                | Enforce minimum 12 characters with uppercase, lowercase, numbers, and symbols. Block known-bad passwords against a blocklist (e.g. Have I Been Pwned API). |
| <b>Modern Hashing</b>                 | Replace MD5crypt (\$1\$) with bcrypt (\$2b\$) or Argon2id. Both are memory-hard and orders of magnitude slower to crack than MD5.                          |
| <b>Shadow File Monitoring</b>         | Add auditd rule: -w /etc/shadow -p r -k shadow_read to alert on any read access to the shadow file outside of expected system processes.                   |
| <b>Privilege Escalation Detection</b> | Monitor for sudo usage by non-admin accounts, SUID binary execution, and unexpected root process spawning — prerequisites for credential dumping.          |
| <b>MFA Enforcement</b>                | Even cracked credentials are useless if MFA is required for all interactive logins — reduces T1078 (Valid Accounts) exploitation impact to near zero.      |
| <b>Account Lockout</b>                | Implement pam_faillock or equivalent to lock accounts after 5 failed attempts, mitigating online brute force attacks against live services.                |